Welcome to your next lesson: Arrays (JS lists).

I'm getting your personalized lesson ready—based on what you've learned, what's coming up, and the best way to build your skills.

Step by step, you’re moving forward.

Ron, let’s connect this lesson to what you already know from Python. 🌱

In Python you’d write a list like:

Python
colors = ["red", "green", "blue"]
In JavaScript, the closest thing is an array, and it looks almost the same:

Javascript
let colors = ["red", "green", "blue"];
Square brackets, commas, ordered collection — just like a Python list.

Quick check (just in your own words):

Does ["red", "green", "blue"] feel clear as “a list of three strings” in JavaScript, or is there anything confusing right away?

Yes, that makes sense

Great 👍

Next tiny step: getting items out of a JavaScript array.

In Python, with

Python
colors = ["red", "green", "blue"]

```
first = colors[0]
first would be "red".


In JavaScript it’s the same idea:

Javascript
let colors = ["red", "green", "blue"];
let first = colors[0];  // "red"
Arrays use zero-based indexing too: index 0 is the first item.

Make sense that colors[0] gives you "red" here?

If yes, I’ll have you try a tiny array + index yourself in
the editor next. 🎯

Yes, that makes sense

Nice. Let’s get you doing this in JavaScript. 🥾


Open the code editor and type this:

Javascript
let playlist = ["Intro", "Verse", "Chorus"];

console.log(playlist[0]);  // first item
console.log(playlist[1]);  // second item
console.log(playlist[2]);  // third item
Then:


Hit Run to see the output.
If it looks good, hit Submit code so I can review it.

After you do that, tell me: which exact line prints "Chorus"?

Javascript
let playlist = ["Intro", "Verse", "Chorus"];

console.log(playlist[0]);  // first item
console.log(playlist[1]);  // second item
console.log(playlist[2]);  // third item
Nice work, Ron — that’s exactly right ✅


The line that prints "Chorus" is:
```

```
Javascript
console.log(playlist[2]);  // third item
Index 2 → third item, because we start counting at 0.

Next step: let’s see if you’re solid on indexes.


Imagine this array:

Javascript
let fruits = ["apple", "banana", "cherry", "date"];
Which index would give you "cherry"?

Just answer with the number (0, 1, 2, or 3).

2

Exactly right 🎯

fruits[2] is "cherry" because:


fruits[0] → "apple"
fruits[1] → "banana"
fruits[2] → "cherry"
fruits[3] → "date"
Now let’s add one new piece: array length.


In JavaScript:

Javascript
let fruits = ["apple", "banana", "cherry", "date"];
console.log(fruits.length);
This will print the number of items in the array (here, 4).


Important: it’s .length, not .length() (it’s a property, not
a function).

Can you open the editor and:


Create this array:
let fruits = ["apple", "banana", "cherry", "date"];
console.log(fruits.length);
```

Run it, then Submit code, and I’ll check it.

Javascript
```
let fruits = ["apple", "banana", "cherry", "date"];
console.log(fruits.length);
```
Perfect — you used .length exactly right, and 4 is the correct output ✅

Quick visual reminder:

Array: ["apple", "banana", "cherry", "date"]
Indexes: 0 1 2 3
.length → 4 (number of items)
Last valid index → length − 1 → 3
Next concept: adding items with push().

In JavaScript:

Javascript
```
let fruits = ["apple", "banana"];
fruits.push("cherry");  // adds to the end
```
After this, fruits is ["apple", "banana", "cherry"].

Does that idea feel clear: push adds one more item to the end of the array?

Yes, that’s clear

Great 👍

Let’s try push() in your own code.

In the editor, write this:

Javascript
```
let playlist = ["Intro", "Verse"];

playlist.push("Chorus");

console.log(playlist);
```

```
console.log(playlist.length);
Then:


Hit Run to see the output.
Hit Submit code so I can review it.

Don’t tell me the output yet — I’ll read it from the run.

Javascript
let playlist = ["Intro", "Verse"];

playlist.push("Chorus");

console.log(playlist);
console.log(playlist.length);
Nice, Ron — that’s exactly what I wanted to see 🎧


playlist became ["Intro", "Verse", "Chorus"]
.length is 3 after the push("Chorus")

So push() both:


changes the array, and
changes the .length.
Next piece: removing the last item with pop().


Example:

Javascript
let playlist = ["Intro", "Verse", "Chorus"];
let last = playlist.pop();  // removes "Chorus" and returns it
After this:


playlist is ["Intro", "Verse"]
last is "Chorus"

Does that behavior make sense: pop() removes the last item
and gives it back to you?

Yes, that makes sense

Perfect 👌
```

Let’s use pop() now.

In the editor, write this:

Javascript
let playlist = ["Intro", "Verse", "Chorus"];

let lastTrack = playlist.pop();

console.log("Playlist now:", playlist);
console.log("Last track was:", lastTrack);
console.log("Length now:", playlist.length);
Then:

Run it.
Hit Submit code so I can see the output.

We’ll use this to double-check you’ve got push, pop, and .length all working together.

Javascript
let playlist = ["Intro", "Verse", "Chorus"];

let lastTrack = playlist.pop();

console.log("Playlist now:", playlist);
console.log("Last track was:", lastTrack);
console.log("Length now:", playlist.length);
Nice job, Ron — this is spot on 🎯

From that run we can see:

playlist after pop() → ["Intro", "Verse"]
lastTrack → "Chorus"
playlist.length → 2

So you’ve now used:

Arrays: ["Intro", "Verse", "Chorus"]
Zero-based indexing: playlist[0], playlist[1], etc.

```
Adding with push()
Removing with pop()
Size with .length
One last small idea: getting the last item using .length.


For:

Javascript
let playlist = ["Intro", "Verse", "Chorus"];
playlist.length is 3
the last index is playlist.length - 1 → 2
so playlist[playlist.length - 1] is "Chorus"

Quick question for you (just answer in chat):


If we have

Javascript
let fruits = ["apple", "banana", "cherry", "date"];
what expression would give the last item using .length?

fruits.length - 1

Nice thinking — that's almost the full idea 😊


fruits.length - 1 gives you the last index, but to get the
actual item you need to use that index on the array:

Javascript
fruits[fruits.length - 1]
So:


fruits.length → 4
fruits.length - 1 → 3
fruits[3] → "date" ✅
Want to try a quick practice in the editor?


Write code that:


Creates this array:
let fruits = ["apple", "banana", "cherry", "date"];
```

Logs the last item using fruits[fruits.length - 1].

Run it, then hit Submit code, and I'll check.

Javascript
let fruits = ["apple", "banana", "cherry", "date"];
console.log(fruits[fruits.length-1]);
Beautiful, Ron — that line is exactly right ✅

Javascript
let fruits = ["apple", "banana", "cherry", "date"];
console.log(fruits[fruits.length - 1]); // "date"
You've now shown you can:

Create arrays and access elements with zero-based indexing
Add/remove items with push() and pop()
Use .length to get the number of items (and to reach the last one)

🎓 That's everything this lesson wanted you to master.

You've got arrays solid for this stage.

Are you ready to wrap up this lesson and move on to the next one, or do you want one more quick practice with push/pop first?

Move to the next lesson

Done 🎉 You've officially completed Arrays (JS lists).

Quick recap of what you can now do in JavaScript:

Create arrays:
let items = ["a", "b", "c"];
Access by zero-based index:
items[0], items[1], etc.
Add to the end with push():
items.push("d");
Remove the last item with pop() and capture it:
let last = items.pop();
Use .length to get how many items there are (and compute the last index with length - 1).

You’re ready for the next lesson 🚀

When you’re back in the Learn tab, just hit Next Lesson and we’ll keep building from here.

If anything about arrays still feels even a bit fuzzy, tell me and I’ll clear it up before you dive on.

How would you rate this lesson?